

Assignment 2 – Case Study

Stage 4 Tutorial Activities

Object-Oriented Design Decisions

Week 7 | 60 minutes

Activity 1 - Encapsulation Review

Class	Protected state / invariant	Public operations
Patient	Patient_name can never be empty. Patient_ID is only given once and can't be changed	Can only read patient name and patient ID
Practitioner	Practitioner_name must never be empty. Practitioner_ID is only given once and can't be named	Can read only practitioner name and practitioner ID
Appointment	Once an appointment is completed or cancelled it can't be changed.	Can cancel, complete, update the time. Status is read only

Activity 2 - Composition or Inheritance?

Appointment and Patient -> ☐ Composition/association: An appointment links to a patient but a patient can be independent and have no appointments.

Appointment and Practitioner -> ☐ Composition/association Reason: As appointment links to a practitioner but a practitioner can be booked separately or have no appointments.

Doctor and Practitioner (hypothetical) -> ☐ Inheritance Reason: the doctor would inherit all the same attributes to a practitioner.

Clinic and Appointment -> ☐ Composition/association Reason: The clinic is what has the practitioners, patient's ad appointments. And the appointment is not a clinic.

Activity 3 - Responsibility Allocation

Who decides whether SCHEDULED can become CANCELLED?

The appointment class holds the cancel code.

Who validates a patient name?

The patient class is where the patient information is approved and stored.

Should Appointment execute SQL? Why?

No as it will make appointment class harder to test. However, it can be kept in a separate part on its own .

Should the UI decide whether a status transition is legal?

No, the UI should only decide on cancel or complete and react if it is successful or an error as it would end up being written in two sections as it will be duplicated in appointment to.

Activity 4 - AI Code Critique

AI generates an Appointment class with public status mutation, SQL inside cancel(), a NotificationManager dependency and inheritance from PatientRecord. Identify at least five design problems and corrections.

Public status mutation: Lets you make the status canceled from any part of the system this can cause anyone to set it to a status. Only let the status change through being cancelled or complete

SQL inside cancel(): Putting the SQL inside the cancel in appointment class puts cancelling and saving it to database in the same section. Keep them separated and in different classes as its two different codes.

NotificationManager dependency: Was not stated in the design that it is needed. To fix this just delete it

Inheritance from PatientRecord: An appointment is not a patient record all it needs to know is the patient information. Don't inherit it.

Cancel sends the message: It sends a message through NotificationManager which it is doing two things at once changing and telling. Focus cancel just on cancelling/changing.

Exit question

Why can code be object-oriented syntactically but still have poor object-oriented design?

As you can use a class but still mess up the important part of the code. Cramming too much into a class or not enough doesn't allow the code to work properly. It being syntactically means the code just uses classes and inheritance but it doesn't mean it is a good object-oriented design.